

Homework 04

Rahaf Alabdullah (rahafaa2)

Question 1: Signal strength and correlated substitutes

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import Lasso
rng = np.random.default_rng(43246)
repetitions = 200
n, p = 120, 10
sigma = 1.0
lambda_grid = np.array([0.08, 0.12, 0.18, 0.24])
results = np.zeros((200, 10, 4))
for i in range(repetitions):
    X_raw = rng.normal(loc=0.0, scale=1.0, size=(n, p))
    X1 = X_raw[:, 0]
    Z = rng.normal(loc=0.0, scale=1.0, size=n)
    X2 = 0.999 * X1 + np.sqrt(1 - 0.999**2) * Z
    X3, X4, X5, X6, X7, X8, X9, X10 = [rng.normal(loc=0.0, scale=1.0, size=n) for _ in range(8)]
    X_ = np.column_stack((X1, X2, X3, X4, X5, X6, X7, X8, X9, X10))
    epsilon = rng.normal(loc=0.0, scale=sigma, size=n)
    Y = 0.15*X1 + 0.15*X2 + 0.30*X3 + 0.15*X4 + epsilon
    x_bar = np.mean(X_, axis=0)
    s = np.sqrt(np.mean((X_ - x_bar) ** 2, axis=0))
    X = X_ - x_bar / s
    y_tilde = Y - np.mean(Y)

    for idx, lam in enumerate(lambda_grid):
        lasso = Lasso(alpha=lam, tol=1e-10, max_iter=10000, fit_intercept=False)
        lasso.fit(X, y_tilde)
        if lasso.n_iter_ >= lasso.max_iter:
            print(f"rep {i}, lambda={lam} did not converge")
            results[i, :, idx] = lasso.coef_ / s

nonzero = np.abs(results) > 1e-8 # shape (200, 10, 4), boolean
selection_freq = nonzero.mean(axis=0) # shape (10, 4): covariate x lambda
mean_abs_slope = np.abs(results).mean(axis=0) # shape (10, 4), zeros included

selection_table = pd.DataFrame(
    selection_freq,
    index=[f"X{j}" for j in range(1, p + 1)],
    columns=[f"lambda_{lam}" for lam in lambda_grid],
)
magnitude_table = pd.DataFrame(
    mean_abs_slope,
    index=[f"X{j}" for j in range(1, p + 1)],
    columns=[f"lambda_{lam}" for lam in lambda_grid],
)
print(selection_table.round(3).to_string())
print(magnitude_table.round(3).to_string())

curve_colors = ["#C84A16", "#9A5B13", "#2F6FB3", "#13294B"]
fig, axes = plt.subplots(1, 2, figsize=(9, 4.4))
for l, (lam, color) in enumerate(zip(lambda_grid, curve_colors)):
    axes[0].plot(
        np.arange(1, p + 1), selection_freq[:, l],
        color=color, linewidth=2, label=f"$\lambda={lam}$",
    )
    axes[1].plot(
        np.arange(1, p + 1), mean_abs_slope[:, l],
        color=color, linewidth=2, label=f"$\lambda={lam}$",
    )
axes[0].set(
    xlabel="Covariate j", ylabel="Selection frequency", ylim=(0, 1)
```

```

)
axes[1].set(xlabel="Covariate j", ylabel=r"Mean $\widehat{\beta}_j$")
for ax in axes:
    ax.legend(frameon=False)
    ax.spines[["top", "right"]].set_visible(False)
fig.tight_layout()
plt.show()

#part b
x1_nonzero = nonzero[:, 0, :]
x2_nonzero = nonzero[:, 1, :]
only_x1 = x1_nonzero & ~x2_nonzero
only_x2 = ~x1_nonzero & x2_nonzero
both = x1_nonzero & x2_nonzero
neither = ~x1_nonzero & ~x2_nonzero
part_b = pd.DataFrame({
    "lambda": lambda_grid,
    "X1": only_x1.mean(axis=0),
    "X2": only_x2.mean(axis=0),
    "both": both.mean(axis=0),
    "neither": neither.mean(axis=0),
})
print(part_b.round(3).to_string(index=False))

#part c
part_c = pd.DataFrame({
    "lambda": lambda_grid,
    "X3": selection_freq[2],
    "X4": selection_freq[3],
    "mean_X5_to_X10": selection_freq[4:10].mean(axis=0),
})
print(part_c.round(3).to_string(index=False))

```

(a) Selection frequency

Covariate	$\lambda=0.08$	$\lambda=0.12$	$\lambda=0.18$	$\lambda=0.24$
X1	0.540	0.525	0.475	0.375
X2	0.500	0.490	0.440	0.380
X3	0.995	0.985	0.880	0.650
X4	0.775	0.645	0.395	0.170
X5	0.385	0.215	0.065	0.010
X6	0.420	0.195	0.035	0.015
X7	0.410	0.245	0.090	0.025
X8	0.380	0.205	0.050	0.005
X9	0.345	0.185	0.050	0.005
X10	0.370	0.170	0.045	0.005

Mean absolute fitted slope

Covariate	$\lambda=0.08$	$\lambda=0.12$	$\lambda=0.18$	$\lambda=0.24$
X1	0.120	0.098	0.066	0.041
X2	0.107	0.088	0.060	0.036
X3	0.214	0.173	0.116	0.070
X4	0.087	0.057	0.026	0.010
X5	0.021	0.010	0.002	0.000
X6	0.019	0.007	0.001	0.000
X7	0.025	0.012	0.004	0.000

Covariate	$\lambda=0.08$	$\lambda=0.12$	$\lambda=0.18$	$\lambda=0.24$
X8	0.019	0.008	0.001	0.000
X9	0.018	0.007	0.001	0.000
X10	0.017	0.007	0.001	0.000

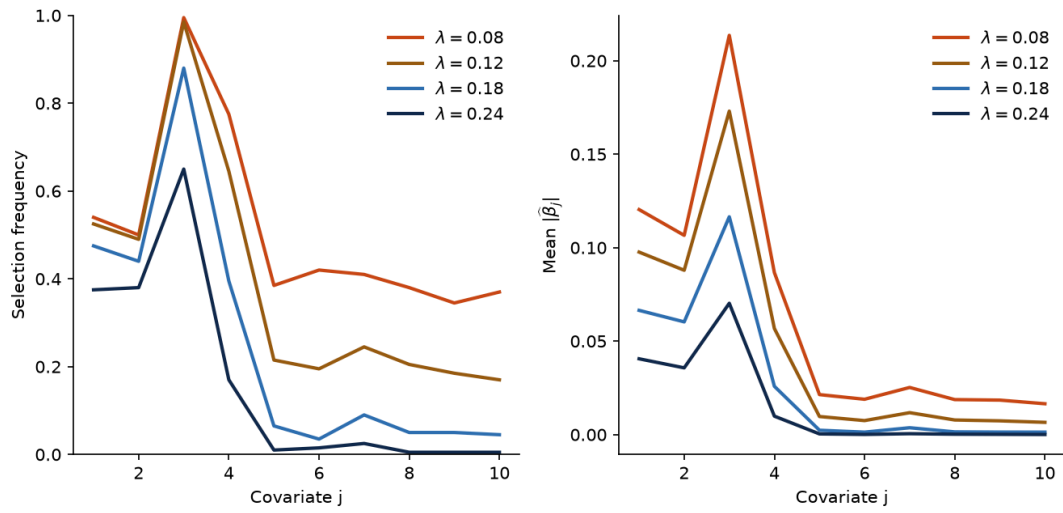


Figure 1: Selection frequency and mean absolute slope by covariate and λ

(b) (X1, X2) joint outcomes

λ	only X1	only X2	both	neither
0.08	0.490	0.450	0.050	0.010
0.12	0.480	0.445	0.045	0.030
0.18	0.460	0.425	0.015	0.100
0.24	0.355	0.360	0.020	0.265

(c) X3, X4 vs. noise covariates (X5-X10 average)

λ	X3	X4	mean(X5..X10)
0.08	0.995	0.775	0.385
0.12	0.985	0.645	0.202
0.18	0.880	0.395	0.056
0.24	0.650	0.170	0.011

Question 2: Elastic net with an equal penalty mix

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import ElasticNet
rng = np.random.default_rng(43246)
repetitions = 200
n, p = 120, 10
sigma = 1.0
lambda_grid = np.array([0.08, 0.12, 0.18, 0.24])
results = np.zeros((200, 10, 4))
for i in range(repetitions):
    X_raw = rng.normal(loc=0.0, scale=1.0, size=(n, p))
    X1 = X_raw[:, 0]
    Z = rng.normal(loc=0.0, scale=1.0, size=n)
    X2 = 0.999 * X1 + np.sqrt(1 - 0.999**2) * Z
    X3, X4, X5, X6, X7, X8, X9, X10 = [rng.normal(loc=0.0, scale=1.0, size=n) for _ in range(8)]
    X_ = np.column_stack((X1, X2, X3, X4, X5, X6, X7, X8, X9, X10))
    epsilon = rng.normal(loc=0.0, scale=sigma, size=n)
    Y = 0.15*X1 + 0.15*X2 + 0.30*X3 + 0.15*X4 + epsilon
    x_bar = np.mean(X_, axis=0)
    s = np.sqrt(np.mean((X_ - x_bar) ** 2, axis=0))
    X = X_ - x_bar / s
    y_tilde = Y - np.mean(Y)

    for idx, lam in enumerate(lambda_grid):
        elastic_net = ElasticNet(alpha=lam, tol=1e-10, max_iter=10000, fit_intercept=False)
        elastic_net.fit(X, y_tilde)
        if elastic_net.n_iter_ >= elastic_net.max_iter:
            print(f"rep {i}, lambda={lam} did not converge")
        results[i, :, idx] = elastic_net.coef_ / s

nonzero = np.abs(results) > 1e-8
selection_freq = nonzero.mean(axis=0)
mean_abs_slope = np.abs(results).mean(axis=0)

selection_table = pd.DataFrame(
    selection_freq,
    index=[f"X{j}" for j in range(1, p + 1)],
    columns=[f"lambda_{lam}" for lam in lambda_grid],
)
print(selection_table.round(3).to_string())

curve_colors = ["#C84A16", "#9A5B13", "#2F6FB3", "#13294B"]
fig, ax = plt.subplots(1, 1, figsize=(9, 4.4))
for l, (lam, color) in enumerate(zip(lambda_grid, curve_colors)):
    ax.plot(
        np.arange(1, p + 1), selection_freq[:, l],
        color=color, linewidth=2, label=f"$\lambda$={lam}$",
    )
ax.set(
    xlabel="Covariate j", ylabel="Selection frequency", ylim=(0, 1)
)
ax.legend(frameon=False)
ax.spines[["top", "right"]].set_visible(False)
fig.tight_layout()
plt.show()

#part b
x1_nonzero = nonzero[:, 0, :]
x2_nonzero = nonzero[:, 1, :]
only_x1 = x1_nonzero & ~x2_nonzero
only_x2 = ~x1_nonzero & x2_nonzero
both = x1_nonzero & x2_nonzero
neither = ~x1_nonzero & ~x2_nonzero
part_b = pd.DataFrame({
    "lambda": lambda_grid,
    "X1": only_x1.mean(axis=0),
    "X2": only_x2.mean(axis=0),
})
```

```

"both": both.mean(axis=0),
"neither": neither.mean(axis=0),
})
print(part_b.round(3).to_string(index=False))

#part c
part_c = pd.DataFrame({
    "lambda": lambda_grid,
    "X3": selection_freq[2],
    "X4": selection_freq[3],
    "mean_X5_to_X10": selection_freq[4:10].mean(axis=0),
})
print(part_c.round(3).to_string(index=False))

```

(a) Selection frequency

Covariate	$\lambda=0.08$	$\lambda=0.12$	$\lambda=0.18$	$\lambda=0.24$
X1	0.970	0.980	0.975	0.955
X2	0.990	1.000	0.980	0.960
X3	1.000	0.995	0.995	0.985
X4	0.905	0.860	0.765	0.655
X5	0.610	0.495	0.340	0.215
X6	0.640	0.540	0.330	0.190
X7	0.690	0.535	0.385	0.245
X8	0.670	0.505	0.330	0.205
X9	0.625	0.465	0.290	0.180
X10	0.690	0.555	0.300	0.175

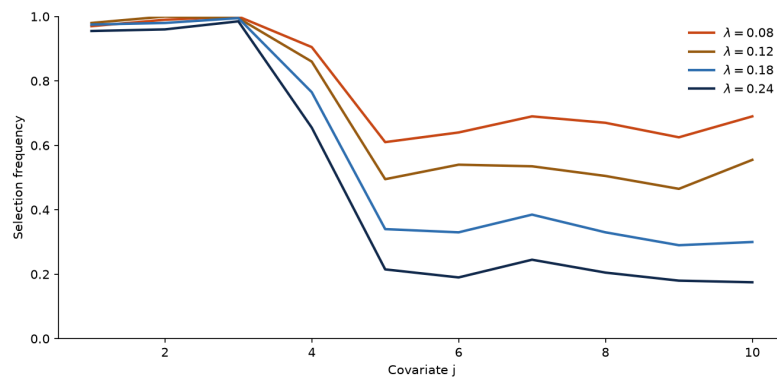


Figure 2: Selection frequency by covariate and λ (elastic net)

(b) (X1, X2) joint outcomes

λ	only X1	only X2	both	neither
0.08	0.010	0.030	0.960	0.000
0.12	0.000	0.020	0.980	0.000
0.18	0.010	0.015	0.965	0.010
0.24	0.010	0.015	0.945	0.030

(c) X3, X4 vs. noise covariates (X5-X10 average)

λ	X3	X4	mean(X5..X10)
0.08	1.000	0.905	0.654
0.12	0.995	0.860	0.516

λ	X3	X4	mean(X5..X10)
0.18	0.995	0.765	0.329
0.24	0.985	0.655	0.202

Question 3: Coordinate descent and the lasso path

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
rng = np.random.default_rng(43247)
n= 100
p=3
sigma= 1.0
X_raw = rng.normal(loc=0.0, scale=1.0, size=(n, p))
Y=( 1.5 + X_raw[:, 0] -2 * X_raw[:, 1] + sigma * rng.normal(loc=0.0, scale=1.0, size=n))

#center X_raw-> each col have zero mean
x_bar = np.mean(X_raw, axis=0)
X = X_raw - x_bar
#Scale so each col average squared value is 1
X = X / np.sqrt(np.mean(X ** 2, axis=0))
#center the response
y_tilde = Y - np.mean(Y)

def soft_threshold_py(a, lam):
    return np.sign(a) * np.maximum(np.abs(a) - lam, 0.0)
lambda_grid = np.array([2.5, 2.0, 1.5, 1.0, 0.6, 0.4, 0.25, 0.15, 0.08, 0.03])
beta = np.zeros(p)
beta_path = np.zeros((len(lambda_grid), p))
cycles = np.zeros(len(lambda_grid))
iterations = 1000
for idx, lam in enumerate(lambda_grid):
    converged = False
    for iteration in range(iterations):
        beta_old = beta.copy()
        for j in range(p):
            r_j= y_tilde - X @ beta + beta[j] * X[:, j]
            a_j=np.mean(X[:, j] * r_j)
            beta[j] = soft_threshold_py(a_j, lam)
        if np.max(np.abs(beta - beta_old)) < 1e-8:
            converged = True
            break
    if not converged:
        print(f"Warning: did not converge for lambda={lam}")
        beta_path[idx]=beta.copy()
        cycles[idx] = iteration + 1

path_table = pd.DataFrame(beta_path, columns=["beta_1", "beta_2", "beta_3"])
path_table.insert(0, "lambda", lambda_grid)
path_table["cycles"] = cycles
print(path_table.round(4).to_string(index=False))

lambda_max = np.max(np.abs(X.T @ y_tilde / n))
print(f"lambda_max = {lambda_max:.4f}")

for j in range(p):
    plt.plot(np.log(lambda_grid), beta_path[:, j], label=f"x_{j+1}")
plt.xlabel("log(lambda)")
plt.ylabel("coefficient")
plt.legend()
plt.show()
beta_ols, *_ = np.linalg.lstsq(X, y_tilde, rcond=None)
print(f"OLS coefficients = {beta_ols}")
```

(a) Coefficient path and convergence

λ	β_1	β_2	β_3	cycles
2.50	0.0000	-0.0000	-0.0000	1
2.00	0.0000	-0.2373	-0.0000	2
1.50	0.0000	-0.7373	-0.0000	2
1.00	0.0000	-1.2373	-0.0000	2

λ	β_1	β_2	β_3	cycles
0.60	0.2553	-1.6183	-0.0000	5
0.40	0.4416	-1.8046	0.0000	5
0.25	0.5812	-1.9442	0.0000	5
0.15	0.6743	-2.0373	0.0000	5
0.08	0.7441	-2.1168	0.0587	7
0.03	0.7963	-2.1807	0.1298	7

All 10 fits converged (largest coefficient change below 10^{-8} within the iteration cap). $\lambda_{\max} = 2.2373$ – consistent with all slopes at $\lambda \geq \lambda_{\max}$ being zero.

(c) Smallest- λ fit vs. OLS

OLS coefficients on the same standardized data: $(0.8276, -2.2191, 0.1724)$, compared to the lasso fit at $\lambda = 0.03$: $(0.7963, -2.1807, 0.1298)$.

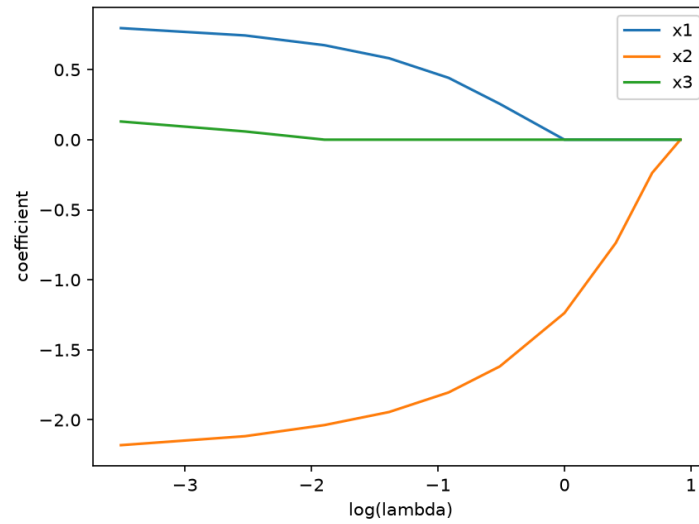


Figure 3: Lasso coefficient path vs. $\log(\lambda)$

Question 4: Tuning and comparing penalized regressions for diabetes prediction

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
diabetes = pd.read_csv("data/diabetes.csv")
folds = pd.read_csv("data/diabetes-split-folds.csv")
diabetes["row_id"] = np.arange(1, len(diabetes) + 1)
merged = diabetes.merge(folds, on="row_id")
train = merged[merged["split"] == "train"]
test = merged[merged["split"] == "test"]

from sklearn.model_selection import PredefinedSplit
ps = PredefinedSplit(test_fold=train["cv_fold"].to_numpy())

from sklearn.linear_model import Lasso, Ridge, ElasticNet
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import GridSearchCV

alphas = np.logspace(-4, 2, 61)
X_train, y_train = train.drop(columns=["row_id", "split", "cv_fold", "y"]), train["y"]
X_test, y_test = test.drop(columns=["row_id", "split", "cv_fold", "y"]), test["y"]

models = {
    "lasso": Lasso(max_iter=100000),
    "ridge": Ridge(),
    "elastic_net": ElasticNet(l1_ratio=0.5, max_iter=100000),
}

results = {}
for name, model in models.items():
    pipe = Pipeline([("scaler", StandardScaler()), ("model", model)])
    grid = GridSearchCV(pipe, {"model__alpha": alphas}, cv=ps, scoring="neg_mean_squared_error")
    grid.fit(X_train, y_train)
    results[name] = grid

    mean_mse = -grid.cv_results_["mean_test_score"]
    se_mse = grid.cv_results_["std_test_score"] / np.sqrt(ps.get_n_splits())
    plt.errorbar(np.log10(alphas), mean_mse, yerr=se_mse)
    plt.title(f"{name} CV MSE vs log10(alpha)")
    plt.xlabel("log10(alpha)")
    plt.ylabel("CV MSE")
    plt.show()

best_name = min(results, key=lambda k: -results[k].best_score_)
best_model = results[best_name].best_estimator_
test_mse = np.mean((best_model.predict(X_test) - y_test) ** 2)
print(f"Best model: {best_name}")
print(f"Best MSE: {test_mse:.4f}")
print(f"Best alpha: {results[best_name].best_params_['model__alpha']:.4f}")
summary_table = pd.DataFrame({
    "Model": list(results.keys()),
    "Best Alpha": [results[name].best_params_["model__alpha"] for name
in results],
    "Best CV MSE": [-results[name].best_score_ for name in results]
})
print(summary_table.round(4).to_string(index=False))
```

Tuning comparison

Model	Best α	CV MSE
lasso	0.1000	3071.275
ridge	0.5012	3073.486
elastic_net	0.0032	3073.449

Selected method: **lasso** ($\alpha = 0.100$). Final test-set MSE (evaluated once): **2732.358**.

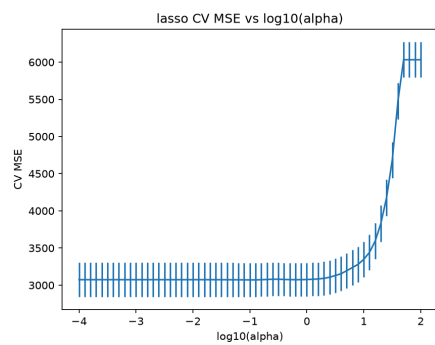


Figure 1: Lasso

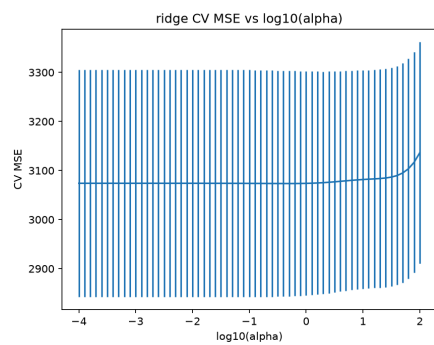


Figure 2: Ridge

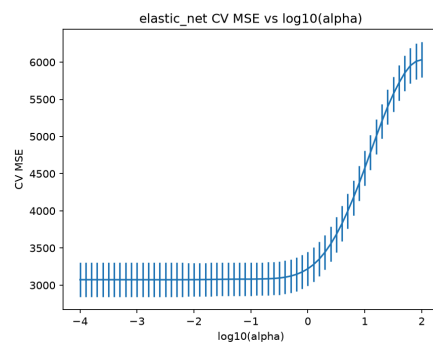


Figure 3: Elastic net